

課題 2：折り目や角を考慮したスムーズシェーディング，Tutte パラメータ化によりテクスチャ座標を計算する実装

氏名：南條佑太
所属・学年：学際科学科 3 年

2026 年 7 月 6 日

1 課題 2-1: 折り目や角を考慮したスムーズシェーディング (smooth)

1.1 コード

```
1 for ( auto& vt : vertices ) {
2     VertexLCirculator vc(vt);
3     std::shared_ptr<HalfedgeL> vh = vc.beginHalfedgeL();
4
5     if ( isCrease(vh) ) vh = vc.prevHalfedgeL();
6     std::shared_ptr<HalfedgeL> start_vh = vh;
7
8     auto norm = std::make_shared<NormalL>();
9     normals.push_back(norm);
10    Eigen::Vector3d sum_n(0, 0, 0);
11
12    do {
13        sum_n += vh->face()->normal();
14        vh->setNormal(norm);
15
16        bool crease_ahead = isCrease(vh);
17        vh = vc.prevHalfedgeL();
```

```

18
19     if ( crease_ahead && (vh != start_vh) ) {
20         norm->point() = sum_n.normalized();
21         norm = std::make_shared<NormalL>();
22         normals.push_back(norm);
23         sum_n = Eigen::Vector3d(0, 0, 0);
24     }
25 } while (vh != start_vh);
26
27 norm->point() = sum_n.normalized();
28 }

```

1.2 プログラムの説明

全頂点を1つずつ処理する。各頂点について、巡回の開始ハーフエッジを取り、それが折り目のエッジなら1つ先の面へ移ってから、その位置を基準点とする。次に最初のグループ用の法線オブジェクトを作って法線リストに登録し、面法線を足し合わせる合計のベクトルを用意する。do-whileでは基準点に戻るまで巡回を続け、各面でその面法線を合計に加え、現在のハーフエッジに現在のグループの法線ポインタを割り当てる。次に越えるエッジが折り目かを記録してから1つ先の面へ進み、先が折り目であり基準点でなければそこがグループの切れ目なので、合計を正規化して法線を確定し、次のグループ用に新しい法線を作ってリストに登録し合計をゼロに戻す。ループを抜けた後、最後のグループの合計を正規化して確定する。

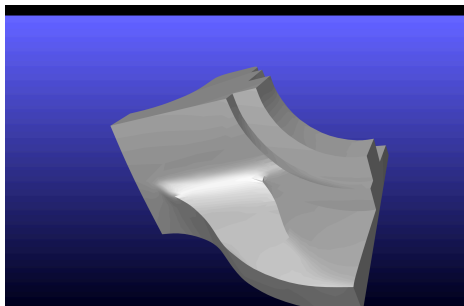


図1 fandisk のスムーズシェーディング結果

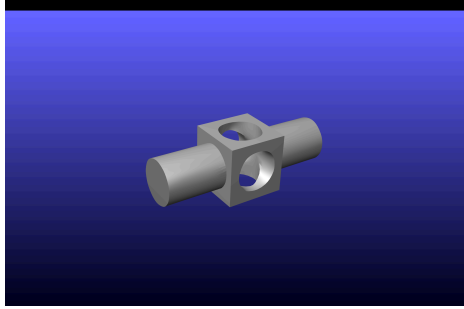


図2 mechapart のスムーズシェーディング結果

2 課題 2-2: Tutte パラメータ化によるテクスチャ座標の計算 (tutteparam)

2.1 前半

```
1  const int nb = static_cast<int>(boundary.size());
2      if (nb < 4) return false;
3
4      const std::array<Eigen::Vector2d, 4> square = {
5          Eigen::Vector2d(0.0, 0.0),
6          Eigen::Vector2d(1.0, 0.0),
7          Eigen::Vector2d(1.0, 1.0),
8          Eigen::Vector2d(0.0, 1.0)};
9
10     const std::vector<double> edge_len = boundaryEdgeLengths(boundary);
11
12     for (int side = 0; side < 4; ++side) {
13         int start = corners[side];
14         int stop = corners[(side + 1) % 4];
15         if (stop <= start) stop += nb;
16
17         double side_len = 0.0;
18         for (int k = start; k < stop; ++k) side_len += edge_len[k % nb];
19         if (side_len < 1.0e-12) return false;
20
21         const Eigen::Vector2d& q0 = square[side];
22         const Eigen::Vector2d& q1 = square[(side + 1) % 4];
23
24         double dist = 0.0;
```

```

25     for (int k = start; k < stop; ++k) {
26         const double t = dist / side_len;
27         const Eigen::Vector2d uv = (1.0 - t) * q0 + t * q1;
28         const int vid = vtx_index_.at(boundary[k % nb]);
29         uv_u_[vid] = uv.x();
30         uv_v_[vid] = uv.y();
31         dist += edge_len[k % nb];
32     }
33 }

```

2.2 プログラムの説明

実装方針に従いコードを作成した。まず境界頂点数 nb を取り、4 未満なら固定 UV を作れないので `false` を返す。正方形の 4 つの角を、左下から反時計回りで UV 空間のベクトルとして定義する。次に `boundaryEdgeLengths` で境界上の各辺の 3D 長さを取得する。そして正方形の 4 辺を `for` 文で順に処理する。`side` 番目の辺は境界配列上で `corners[side]` から `corners[(side+1)%4]` までの区間に対応し、最後の辺は末尾から先頭へ戻るため、 nb を足して連続区間として扱えるようにしている。各辺ではまずその区間の 3D 長さの合計 `side_len` を求め、これが 0 に近い場合はゼロ割りを避けるため `false` を返す。続いて UV 空間での始点 q_0 と終点 q_1 を取る。区間内の各頂点について、区間の始点からその頂点までの累積 3D 距離 $dist$ を辺全体の長さで割った割合 $t = \frac{dist}{side_len}$ を求め、 $uv = (1 - t)q_0 + tq_1$ で正方形の辺上を線形補間した点をその頂点の固定 UV として `uv_u_`・`uv_v_` に書き込む。その後 $dist$ を辺長ぶん進めて次の頂点へ移り、辺の端まで繰り返す。

2.3 後半

```

1  for (int i = 0; i < n; ++i) {
2      if (is_boundary_[i]) {
3          triplets.emplace_back(i, i, 1.0);
4          bu[i] = uv_u_[i];
5          bv[i] = uv_v_[i];
6      } else {
7          VertexLCirculator vc(vtx_order_[i]);
8          std::shared_ptr<VertexL> nvt = vc.beginVertexL();
9          std::shared_ptr<VertexL> start_nvt = nvt;
10
11          std::vector<int> neighbors;
12          do {

```

```

13     neighbors.push_back(vtx_index_.at(nvt));
14     nvt = vc.nextVertexL();
15 } while ((nvt != start_nvt) && (nvt != nullptr));
16
17     const int d = static_cast<int>(neighbors.size());
18     if (d == 0) return false;
19
20     const double lambda = 1.0 / static_cast<double>(d);
21     triplets.emplace_back(i, i, 1.0);
22     for (int j : neighbors) {
23         triplets.emplace_back(i, j, -lambda);
24     }
25 }
26 }

```

2.4 プログラムの説明

実装方針にしたがい作成し、頂点番号 i の順に各行を処理した。境界頂点の場合は Dirichlet 条件として、その頂点の UV 座標を `mapBoundaryToSquare` で固定した値に等しくする式を立てる。具体的には行列の対角に $A(i, i) = 1$ を置き、右辺の $\text{bu}[i] \cdot \text{bv}[i]$ に固定 UV を入れる。これで i 行の方程式は $u_i = \text{uv_u_}[i], v_i = \text{uv_v_}[i]$ となり、その頂点の値が動かないよう固定される。内部頂点の場合は Tutte の平均値条件を入れる。頂点 i を隣接頂点の平均位置に配置したいため、条件は

$$p_i = \frac{1}{d_i} \sum_{j \in N(i)} p_j \quad (1)$$

である。 d_i は頂点 i の隣接頂点数、 j は i の隣接頂点群 $N(i)$ 。右辺を左辺へ移項して

$$p_i - \frac{1}{d_i} \sum_{j \in N(i)} p_j = 0 \quad (2)$$

とする。この左辺を行列の i 行として表すと、対角成分は $A(i, i) = 1$ 、各隣接頂点 j に対応する列については $A(i, j) = -\lambda_{ij}$ となる。ここで $\lambda_{ij} = \frac{1}{d_i}$ である。右辺ベクトル $\text{bu}[i] \cdot \text{bv}[i]$ の値は 0 のままにする。 $\lambda_{ij} = \frac{1}{d_i}$ は各隣接頂点へ等しい重みを与える単純平均を意味している。

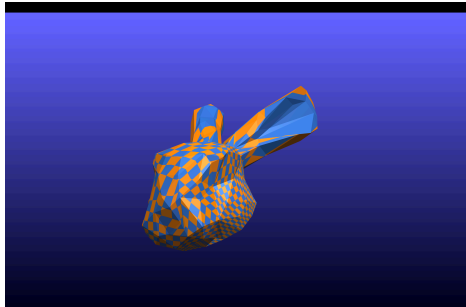


図 3 bunnyhead の Tutte パラメータ化結果．テクスチャマッピング

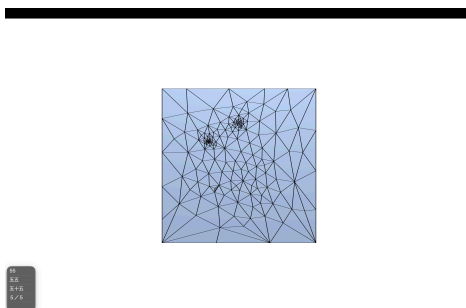


図 4 bunnyhead の Tutte パラメータ化結果．パラメータ表示

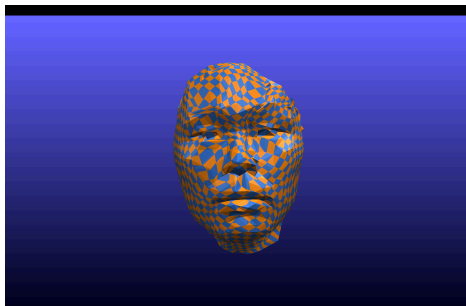


図 5 kanaif の Tutte パラメータ化結果．テクスチャマッピング

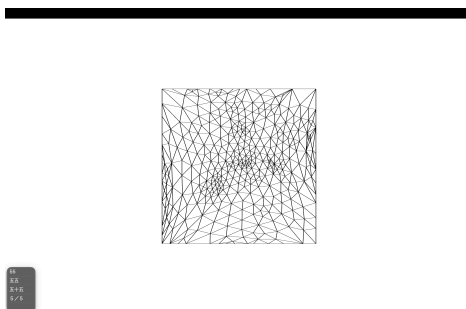


図 6 kanaif の Tutte パラメータ化結果．パラメータ表示